Task Collaboration Management API

Backend Endpoint Documentation

Version 1.0 | April 2026

This document provides complete API endpoint documentation for the Workspace Management platform, covering Users, Workspaces, Projects, Tasks, Messages, and Comments.

Table Of Contents:

#General Information

1. Authentication
2. Users
3. Workspace
4. Workspace Members
5. Projects
6. Tasks
7. Task Assignees
8. Messages
9. Comments
10. Error References

Information

Tech Stack Used:

- **Backend:** Node.js + Express
- **Database:** MySQL
- **DB Driver:** mysql2
- **Authentication:** JWT (Bearer Token)
- **Password Hashing:** bcrypt

Base URL: https://api.workspaceapp.com/v1

Test URL: http://localhost:5000/api

1. Authentication

All API requests require a Bearer token in the Authorization header. Obtain tokens via the login endpoint.

Headers

```
Authorization: Bearer <access_token>
Content-Type: "application/json"
```

POST /auth/register

Register a new user account.

Request Body

Parameter	Type	Required	Description
`name`	string	Yes	Full name of the user
`email`	string	Yes	Valid email address
`username`	string	Yes	Unique username (3-30 chars)
`password`	string	Yes	Min 8 chars, 1 uppercase, 1 number

Response (201 Created)

```
{
   status: "success",
   data: {
       "id": 1,
       "name": "anil",
       "email": "anil@example.com",
       "username": "anil",
       "token": "eyJhbGciOiJIUzI1NiIs..."
   }
}
```

Error Responses

Code	Description
`400`	Validation error (missing fields, weak password)
`409`	Email or username already exists

POST /auth/login

Authenticate and receive an access token.

Request Body

Parameter	Type	Required	Description
`email`	string	Yes	Registered email
`password`	string	Yes	Account password

Response (200 OK)

```
{
   status : “success”,
   data: {
       "token": "eyJhbGciOiJIUzI1NiIs...",
       "user": { "id": 1, "name": "anil", "username": "anil" }
   }
}
```

Error Responses

Code	Description
`401`	Invalid email or password
`429`	Too many login attempts(additional)

2. Users

GET /users/:id

Retrieve a user profile by ID.

Path Parameters

Parameter	Type	Required	Description
id	integer	Yes	User ID

Response (200 OK)

```
{
   status : “success”,
   data: {
       "id": 1,
       "name": "anil",
       "email": "anil@example.com",
       "username": "anil",
       "created_at": "2026-01-15T10:30:00Z"
       }

}
```

Error Responses

Code	Description
404	User not found

PUT /users/:id

Update user profile information.

Request Body

Parameter	Type	Required	Description
name	string	No	Updated name
email	string	No	Updated email
username	string	No	Updated username

Response (200 OK)

```
{
  status : “success”,
  data: {
       "id": 1,
       "name": "anil new",
       "email": "anil.new@example.com",
       "username": "anil_updated",
       "created_at": "2026-01-15T10:30:00Z"
       }
}
```

DELETE /users/:id

Delete a user account and all associated data.

Response (204 No Content)

Empty response body on success.

Error Responses

Code	Description
403	Not authorized to delete this user
404	User not found

3. Workspaces

POST /workspaces

Create a new workspace. The authenticated user becomes the owner.

Request Body

Parameter	Type	Required	Description
name	string	Yes	Workspace name (3-100 chars)

Response (201 Created)

```
{
  status : "success",
  data: {
       "id": 10,
       "name": "Engineering Team",
       "created_by": 1,
       "created_at": "2026-04-15T09:00:00Z"
       }

}
```

GET /workspaces

List all workspaces the authenticated user is a member of.

Query Parameters

Parameter	Type	Required	Description
page	integer	No	Page number (default: 1)(optional)
limit	integer	No	Items per page (default: 20, max: 100)

Response (200 OK)

```
{
  "data": [
    {
       "id": 10, "name": "Engineering Team",
       "role": "admin",
       "created_at": "2026-04-15T09:00:00Z"
    }
  ],
  "pagination": {
       "page": 1,
       "limit": 20,
       "total": 3
       }
}
```

GET /workspaces/:id

Get workspace details including member count.

Response (200 OK)

```
{
  status : “success”,
  data: {
       "id": 10,
       "name": "Engineering Team",
       "created_by": 1,
       "member_count": 8,
       "project_count": 3,
       "created_at": "2026-04-15T09:00:00Z"
       }


}
```

PUT /workspaces/:id

Update workspace details. Requires admin role.

Request Body

Parameter	Type	Required	Description
name	string	Yes	Updated workspace name

Response (200 OK)

```
{
  status : “success”,
  data: {
       “MESSAGE”:”UPDATED SUCCESSFULLY”
  }
}
```

DELETE /workspaces/:id

Delete workspace and all its projects, tasks, and messages. Requires owner role.

Response (204 No Content)

4. Workspace Members

POST /workspaces/:id/members

Add a user to the workspace. Requires admin role.

Request Body

Parameter	Type	Required	Description
`user_id`	integer	Yes	ID of user to add
`role`	string	No	Role: admin, member, viewer (default: member)

Response (201 Created)

```
{
  status : "success",
  data: {
       "id": 50,
       "user_id": 5,
       "workspace_id": 10,
       "role": "member",
       "joined_at": "2026-04-15T10:00:00Z"
       }

}
```

Error Responses

Code	Description
`400`	User is already a member
`403`	Not authorized (admin required)
`404`	User or workspace not found

GET /workspaces/:id/members

List all members of a workspace.

Response (200 OK)

```
{
  "data": [
    { "id": 50, "user_id": 5,
       "name": "Jane Smith",
       "role": "member",
       "joined_at": "2026-04-15T10:00:00Z"
       }
  ]
}
```

PATCH /workspaces/:workspace_id/members/:member_id

Update a member role. Requires admin role.

Request Body

Parameter	Type	Required	Description
`role`	string	Yes	New role: admin, member, viewer

`DELETE /workspaces/:workspace_id/members/:member_id`

Remove a member from workspace. Requires admin role.

5. Projects

POST /workspaces/:workspace_id/projects

Create a new project within a workspace.

Request Body

Parameter	Type	Required	Description
name	string	Yes	Project name (3-100 chars)

Response (201 Created)

```
{
  status : "success",
  data: {
       "id": 20,
       "name": "Website Redesign",
       "workspace_id": 10,
       "created_by": 1,
       "created_at": "2026-04-15T11:00:00Z"
       }


}
```

GET /workspaces/:workspace_id/projects

List all projects in a workspace.

GET /projects/:id

Get project details with task summary.

Response (200 OK)

```
{
  status : "success",
  data:  {
         "id": 20,
         "name": "Website Redesign",
         "workspace_id": 10,
         "created_by": 1,
         "task_count": { "total": 15, "todo": 5, "in_progress": 4, "done": 6 },
         "created_at": "2026-04-15T11:00:00Z"
       }

}
```

PUT /projects/:id

Update project details.

`DELETE /projects/:id`

Delete project and all its tasks. Requires admin role in workspace.

6. Tasks

POST /projects/:project_id/tasks

Create a new task in a project.

Request Body

Parameter	Type	Required	Description
title	string	Yes	Task title (3-200 chars)
description	string	No	Detailed task description
status	string	No	todo, in_progress, review, done (default: todo)
priority	string	No	low, medium, high, urgent (default: medium)
due_date	datetime	No	ISO 8601 due date
assignee_ids	integer[]	No	Array of user IDs to assign

Response (201 Created)

```
{
  status : "success",
  data: {
         "id": 100,
         "title": "Design homepage mockup",
         "description": "Create wireframes for the new homepage",
         "project_id": 20,
         "created_by": 1,
         "status": "todo",
         "priority": "high",
         "due_date": "2026-04-20T00:00:00Z",
         "assignees": [{ "id": 5, "name": "Jane Smith" }],
         "created_at": "2026-04-15T12:00:00Z"
       }

}
```

GET /projects/:project_id/tasks

List all tasks in a project with filtering and sorting.

Query Parameters

Parameter	Type	Required	Description
status	string	No	Filter by status (comma-separated)
priority	string	No	Filter by priority
assignee_id	integer	No	Filter by assignee
sort	string	No	Sort field: created_at, due_date, priority
order	string	No	asc or desc (default: desc)

page	integer	No	Page number
limit	integer	No	Items per page (default: 20)

GET /tasks/:id

Get full task details including assignees and comment count.

PUT/tasks/:id

Update task fields. Only provided fields are updated.

Request Body

Parameter	Type	Required	Description
title	string	No	Updated title
description	string	No	Updated description
status	string	No	Updated status
priority	string	No	Updated priority
due_date	datetime	No	Updated due date

Response (200 OK)

```
{
  status : "success",
  data: {
       "MESSAGE":"UPDATED SUCCESSFULLY"
  }
}
```

DELETE /tasks/:id

Delete a task and its comments/assignees.

7. Task Assignees

`POST /tasks/:task_id/assignees`

Assign a user to a task.

Request Body

Parameter	Type	Required	Description
`user_id`	integer	Yes	ID of user to assign

RE

Error Responses

Code	Description
`400`	User already assigned to this task
`404`	Task or user not found

`GET /tasks/:task_id/assignees`

List all users assigned to a task.

`DELETE /tasks/:task_id/assignees/:user_id`

Remove a user from a task.

8. Messages

POST /workspaces/:workspace_id/messages

Send a message in a workspace channel.

Request Body

Parameter	Type	Required	Description
content	string	Yes	Message content (1-5000 chars)

Response (201 Created)

```
{
  "id": 200,
  "workspace_id": 10,
  "sender_id": 1,
  "sender_name": "John Doe",
  "content": "Hey team, the sprint starts tomorrow!",
  "created_at": "2026-04-15T14:00:00Z"
}
```

GET /workspaces/:workspace_id/messages

Retrieve messages in a workspace with pagination (newest first).

Query Parameters

Parameter	Type	Required	Description
before	datetime	No	Get messages before this timestamp (cursor)
limit	integer	No	Messages to return (default: 50, max: 100)

DELETE /messages/:id

Delete a message. Only sender or workspace admin can delete.

9. Comments

POST /tasks/:task_id/comments

Add a comment to a task.

Request Body

Parameter	Type	Required	Description
content	string	Yes	Comment text (1-2000 chars)

Response (201 Created)

```
{
  "id": 300,
  "task_id": 100,
  "user_id": 1,
  "user_name": "John Doe",
  "content": "Mockup draft is ready for review.",
  "created_at": "2026-04-15T15:00:00Z"
}
```

GET /tasks/:task_id/comments

List all comments on a task, ordered by creation time.

PUT /comments/:id(OPTIONAL)

Edit a comment. Only the author can edit.

Request Body

Parameter	Type	Required	Description
content	string	Yes	Updated comment text

DELETE /comments/:id

Delete a comment. Only author or workspace admin can delete.

10. Error Reference

All errors follow a standard format:

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Name is required",
    "details": [{ "field": "name", "message": "Must be 3-100 characters" }]
  }
}
```

Standard HTTP Status Codes

Code	Description
200	OK - Request succeeded
201	Created - Resource created successfully
204	No Content - Resource deleted successfully
400	Bad Request - Validation error or malformed request
401	Unauthorized - Missing or invalid authentication token
403	Forbidden - Insufficient permissions
404	Not Found - Resource does not exist
409	Conflict - Resource already exists (duplicate)
429	Too Many Requests - Rate limit exceeded
500	Internal Server Error - Unexpected server failure